

The REST principles and HATEOAS: history and perspectives

Michael Mrissa

Introduction

- The Web
 - Document sharing platform
 - Script languages
 - Application platform
- Technologies
 - URI, HTTP, structured data (JSON, etc.)
 - Hyperlinks
- Web APIs and REST
 - Started with practice (90's, last century)
 - Theory came later (2000, Roy Fielding thesis) [1]

Underlying Principles of the Web [2]

- Everything is a resource
 - Identified with a URI
- Unique API
 - GET PUT POST DELETE...
- Stateless protocol
 - Self-contained message
- Hypermedia
 - Principle of partial understanding
- Properties
 - Scalability
 - Genericity
 - Independent deployment
 - Intermediary components for lower latency (cache)
 - Overall better performance

HATEOAS – the main idea

- Hypertext As The Engine Of Application State
 - See an application as a finite state machine
 - Where links are transitions
- The application provides the links
 - Clients do not need application-specific knowledge
 - Updates are transparent
- So, why didn't it get adopted?

HATEOAS – reasons for failure

- Technologically complex
 - Knowledge of the links
 - Moving from static to dynamic processes
 - Maintenance costs vs ROI
- Lack of incentive: what is the ROI for the service provider?
 - Lack of economical models
 - Lack of openness & collaboration in a competitive environment
- Removes some flexibility on the user-side
 - Users should remain in control
 - Current options are manually selected but unlimited

Limited literature

- [3] service wrapper to add HATEOAS links (3rd party service)
 - No modification of the legacy service
 - Static design of the application
 - Still server-side knowledge
- [4] Another service wrapper
 - Parses OpenAPI descriptions
 - Generates links and links to relevant responses
 - Towards a dynamic approach, but still server-side
 - Bound to one API
- [5] Web links in JSON using “href” as key
 - Supports additional description (link type, language, value)
 - Only an extension to web links
 - Makes navigation richer, clients need to be coded to exploit it
- [6] <https://ionspec.org/>

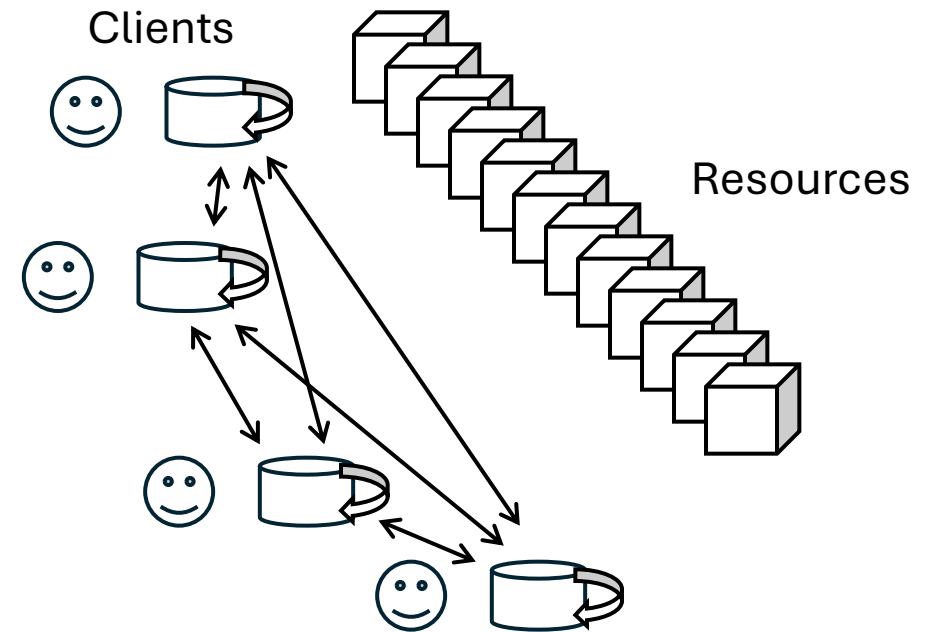
A vision for a modern HATEOAS

- Knowledge about links
 - Should be client-side
- Advantages
 - No server involved
 - Better privacy/security
 - Access to browsing history
 - Naturally cross-API
 - Extensible & natively Web-scale
- Disadvantages
 - You tell me
- Next-gen
 - “Navigation communities”
 - Clients help each other
 - Compromise
 - Between mainstream & diversity
 - Production of general knowledge
 - From individual behavior
 - To general patterns
- Disadvantages
 - You tell me

A vision for a modern HATEOAS

Questions

- Knowledge design
 - Navigation patterns
- Link design
 - Enriched links (cf Ion)
- Access control
 - Defining user profiles
- Sharing protection
 - Many techniques to explore



Conclusion and discussion

- Defining success
 - of HATEOAS, and in general
 - Facts and judgment
 - “failure” ? => learning
 - Assessing the lack of adoption
 - Clarity about requirements
- Web evolution
 - Requirements-driven
 - Tech-dependent
 - Not only science

Thank you for your attention
Questions?

References

- [1] Fielding, R. T. Architectural Styles and the Design of Network-based Software Architectures. Doctoral dissertation, University of California, Irvine, 2000.
- [2] Webber, J., Parastatidis, S., and Robinson, I. REST in Practice: Hypermedia and Systems Architecture. O'Reilly Media, Sebastopol, CA, USA, 2010.
- [3] Liskin O, Singer L, Schneider K. Teaching old services new tricks: adding HATEOAS support as an afterthought. In Proceedings of the Second International Workshop on RESTful Design 2011 Mar 28 (pp. 3-10).
- [4] Kotstein, S., Decker, C. (2020). Navigational Support for Non HATEOAS-Compliant Web-Based APIs. In: Dustdar, S. (eds) Service-Oriented Computing. SummerSOC 2020. Communications in Computer and Information Science, vol 1310. Springer, Cham.
- [5] <https://ionspec.org/> - accessed on the 15.12.2024